# 4. Manual del simulador Flutter

## 4.1. Descripción

El **simulador** es un **servidor ADS** escrito en Dart/Flutter que emula un PLC

TwinCAT. No es un cliente: escucha conexiones TCP y responde a las peticiones ADS

de `LibPhithon` (Python) y `LibC#` (C#).

## 4.2. Requisitos

- SDK de Flutter (incluye Dart). El modo headless solo necesita Dart.
- Para la interfaz gráfica: un destino de escritorio (`windows`, `linux` o `macos`).

## 4.3. Ejecución

### Modo headless (sin interfaz, solo Dart)

```
cd simulador_flutter
dart bin/server.dart            # puerto 48898
dart bin/server.dart 48000      # puerto personalizado
```

### Modo con interfaz (Flutter)

```
cd simulador_flutter
flutter run -d windows
```

## 4.4. Configuración de red

| Parámetro | Valor |
|---|---|
| Dirección TCP | `127.0.0.1:48898` (configurable) |
| AmsNetId | `127.0.0.1.1.1` |
| Puerto AMS (runtime PLC) | `851` |

Los clientes deben conectar su socket TCP a `48898` y dirigir sus cabeceras AMS al

NetId `127.0.0.1.1.1` y puerto AMS `851`.

## 4.5. Variables simuladas

| Símbolo | Tipo | Valor inicial | Comportamiento |
|---|---|---|---|
| `MAIN.bStart` | BOOL | `false` | Escritura libre |
| `MAIN.bStop` | BOOL | `true` | Escritura libre |
| `MAIN.nCounter` | UINT | `0` | Se incrementa cada |

| | | | |
|---|---|---|---|
| | | | 500 ms en estado RUN |
| `MAIN.iLevel` | INT | `100` | Escritura libre |
| `MAIN.fTemp` | REAL | `21.5` | Escritura libre |
| `MAIN.dPosition` | LREAL | `1.2345` | Escritura libre |
| `MAIN.sMessage` | STRING(80) | `"Hola ADS"` | Escritura libre |
| `MAIN.arrValues` | ARRAY[0..3] OF INT | `[1, 2, 3, 4]` | Escritura libre |

## 4.6. Comandos ADS implementados

| **Comando** | **Descripción** |
|---|---|
| `READ_DEVICE_INFO` (0x0001) | Devuelve versión 3.1 y nombre `ADS-Simulator`. |
| `READ_STATE` (0x0004) | Devuelve el estado ADS y de dispositivo. |
| `WRITE_CONTROL` (0x0005) | Cambia el estado (RUN/STOP/RESET…). |
| `READ/WRITE` (0x0002/0x0003) | Por nombre (0xF005), por handle (0xF004), handle-por-nombre (0xF003), info de símbolo (0xF006), tabla de símbolos (0xF000), imagen de proceso (0xF020/0xF030), y release de handle (0xF014). |
| `READ_WRITE` (0x0009) | Lee y escribe en una transacción. |
| `SUMUP_READ` (0xF080) | Lectura múltiple. |
| `SUMUP_WRITE` (0xF081) | Escritura múltiple. |
| `ADD/DEL/DEVICE_NOTIFICATION` | Notificaciones por cambio y por ciclo. |

## 4.7. Notificaciones

- Para suscribirse a una variable por nombre, el cliente primero consulta

`SYM_INFOBYNAME` (obtiene `index_group=0xF005` e `index_offset=<índice>`) y luego

llama a `ADD_DEVICE_NOTIFICATION` con ese par índice/offset.

- El modo **on change** (por defecto) entrega un stamp cuando el valor cambia.
- El modo **cycle** entrega stamps periódicamente según `cycle_time`.

## 4.8. Estructura de archivos

| **Archivo** | **Qué hace** |
|---|---|
| `bin/server.dart` | Servidor headless (Dart puro). |
| `lib/main.dart` | Interfaz gráfica Flutter. |
| `lib/ads/ams_header.dart` | Encode/decode de frame AMS/TCP y cabecera AMS. |
| `lib/ads/constants.dart` | Comandos, grupos de índice, estados, |

| | |
|---|---|
| | puertos y errores. |
| `lib/ads/datatypes.dart` | Codificación de tipos PLC. |
| `lib/simulator/symbol_table.dart` | Tabla de símbolos y handles. |
| `lib/simulator/device.dart` | Estado del PLC, variables y notificaciones. |
| `lib/simulator/ads_server.dart` | Servidor TCP y despacho de comandos. |

## 4.9. Interfaz gráfica

La UI permite:

- **Iniciar/Detener** el servidor.
- **Cambiar el puerto TCP** (antes de iniciar).
- Ver la **tabla de variables** en vivo (nombre, tipo y valor; `MAIN.nCounter`

se actualiza automáticamente).

- **Editar** cada variable (campo de texto + botón de escritura).
- Consultar el **registro** de conexiones y errores.